

Assignment 1: Web Scraping and Name Entity Recognition

Jackie McGinley

March 30, 2026

Instructions:

Web scraping is powerful and useful technique for collecting public data from the internet. For this assignment, you will scrape the Georgetown University wikipedia web page and use named entity recognition (NER) to extract named entities from the page.

You must prepare a Jupyter notebook that contains all necessary code as well as Markdown text to explain each question and step of the process below, how it works, and what is happening. You will save the notebook as an HTML report and submit it to Canvas. The script should run without errors. You should use tables and/or visualizations to show your results when appropriate.

Question 1: Review the source code of the webpage using the inspector tool of your browser. List three types of HTML tags you found on the page and explain what elements of the page are created with those tags.

Answer 1:

After Reviewing the source code of the Georgetown University Wikipedia webpage, the HTML tags were familiar but below are three that stood out immediately to me:

1. The first tag that stuck out to me was `< a >`, this tag lets the user click and go somewhere like another webpage, another section on the same page, etc. For example, `< a href="/wiki/Main_Page" class="mw-logo" >`, will allow the user to click on the Wiki log in the top left hand corner of the page and it will take you to the Wiki home page. This type of tag is a text / content tag, specifically creating a link/anchor.

2. The next tag that stuck out to me was `< div >` because it appears many, many times in the source code. This tag is a block level element that will take up the full width of the page. For example, `< div class = "vector-header-container" >` takes up the top part of the page, where it has the little hamburger, the wiki logo, search bar, etc. This type of tag is a layout / container tag, creating a block container.

3. The third tag that stuck out to me was `< button >` because there are a couple of buttons on this webpage, but also because the search button is right at the top of the page. For example, `< button class="cdx-button cdx-search-input__end-button" > Search < /button >`, which is actually where the search button is getting referenced / created. This type of tag is a form tag.

Question 2: Issue a request for the webpage using the requests library and display the HTTP response code for the request. What is the response status code and what does it indicate? What do status codes in the 400s mean? What do status codes in the 500s mean?

Answer 2:

```
In [11]: import sys
!{sys.executable} -m pip install spacy
!{sys.executable} -m spacy download en_core_web_lg
```

Requirement already satisfied: spacy in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (3.8.14)

Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from spacy) (3.0.12)

Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from spacy) (1.0.5)

Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from spacy) (1.0.15)

Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from spacy) (2.0.13)

Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from spacy) (3.0.13)

Requirement already satisfied: thinc<8.4.0,>=8.3.12 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from spacy) (8.3.13)

Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from spacy) (1.1.3)

Requirement already satisfied: srsly<3.0.0,>=2.5.3 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from spacy) (2.5.3)

Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from spacy) (2.0.10)

Requirement already satisfied: weasel<2.0.0,>=1.0.0 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from spacy) (1.0.0)

Requirement already satisfied: confection<2.0.0,>=1.3.2 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from spacy) (1.3.3)

Requirement already satisfied: typer<1.0.0,>=0.3.0 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from spacy) (0.9.0)

Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from spacy) (4.67.1)

Requirement already satisfied: numpy>=1.19.0 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from spacy) (2.2.5)

Requirement already satisfied: requests<3.0.0,>=2.13.0 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from spacy) (2.32.3)

Requirement already satisfied: pydantic<3.0.0,>=2.0.0 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from spacy) (2.10.3)

Requirement already satisfied: jinja2 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from spacy) (3.1.6)

Requirement already satisfied: setuptools in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from spacy) (80.9.0)

Requirement already satisfied: packaging>=20.0 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from spacy) (24.2)

Requirement already satisfied: annotated-types>=0.6.0 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from pydantic<3.0.0,>=2.0.0->spacy) (0.6.0)

Requirement already satisfied: pydantic-core==2.27.1 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from pydantic<3.0.0,>=2.0.0->spacy) (2.27.1)

Requirement already satisfied: typing-extensions>=4.12.2 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from pydantic<3.0.0,>=2.0.0->spacy) (4.12.2)

Requirement already satisfied: charset-normalizer<4,>=2 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from requests<3.0.0,>=2.13.0->spacy) (3.3.2)

Requirement already satisfied: idna<4,>=2.5 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from requests<3.0.0,>=2.13.0->spacy) (3.7)

Requirement already satisfied: urllib3<3,>=1.21.1 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from requests<3.0.0,>=2.13.0->spacy) (2.3.0)

Requirement already satisfied: certifi>=2017.4.17 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from requests<3.0.0,>=2.13.0->spacy) (2025.10.5)

Requirement already satisfied: blis<1.4.0,>=1.3.0 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from thinc<8.4.0,>=8.3.12->spacy) (1.3.3)

Requirement already satisfied: click<9.0.0,>=7.1.1 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from typer<1.0.0,>=0.3.0->spacy) (8.1.8)

Requirement already satisfied: cloudpathlib>=0.7.0 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from weasel<2.0.0,>=1.0.0->spacy) (0.23.0)

Requirement already satisfied: smart-open>=5.2.1 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from weasel<2.0.0,>=1.0.0->spacy) (7.5.1)

Requirement already satisfied: httpx>=0.24.0 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from weasel<2.0.0,>=1.0.0->spacy) (0.28.1)

Requirement already satisfied: anyio in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from httpx>=0.24.0->weasel<2.0.0,>=1.0.0->spacy) (4.7.0)

Requirement already satisfied: httpcore==1.* in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from httpx>=0.24.0->weasel<2.0.0,>=1.0.0->spacy) (1.0.9)

Requirement already satisfied: h11>=0.16 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from httpcore==1.*->httpx>=0.24.0->weasel<2.0.0,>=1.0.0->spacy) (0.16.0)

Requirement already satisfied: wrapt in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from smart-open>=5.2.1->weasel<2.0.0,>=1.0.0->spacy) (1.17.0)

Requirement already satisfied: sniffio>=1.1 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from anyio->httpx>=0.24.0->weasel<2.0.0,>=1.0.0->spacy) (1.3.0)

Requirement already satisfied: MarkupSafe>=2.0 in /Users/jacquelinemcginley/opt/anaconda3/lib/python3.13/site-packages (from jinja2->spacy) (3.0.2)

Collecting en-core-web-lg==3.8.0

Using cached https://github.com/explosion/spacy-models/releases/download/en_core_web_lg-3.8.0/en_core_web_lg-3.8.0-py3-none-any.whl (400.7 MB)

✓ Download and installation successful

You can now load the package via `spacy.load('en_core_web_lg')`

```
In [12]: import spacy
         spacy.load("en_core_web_lg")
```

```
Out[12]: <spacy.lang.en.English at 0x3328302d0>
```

```
In [13]: import requests
         from bs4 import BeautifulSoup
         import pandas as pd
```

```
In [14]: url = "https://en.wikipedia.org/wiki/Georgetown_University"

         headers = {
             "User-Agent" : "jm3882@georgetown.edu"
         }
         response = requests.get(url, headers = headers)
```

```
print("Status Code:", response.status_code)
```

Status Code: 200

Status Code 200: means a client’s request to a server has been successfully received, understood, and processed

Status Code in 400s: generally mean Bad Request, the server can't understand the request which could be for a number of reasons such as a broken url or invalid syntext

Status Code in 500s: generally referred to as Server Error Response. they can occur due to the server crashing or having a bug, the server didn't respond in time, etc.

Question 3: Extract all raw text contained in paragraph tags on the web page. Store the results in single cell in a pandas DataFrame.

Answer 3:

```
In [15]: soup = BeautifulSoup(response.text, "lxml")

p_text = [p.text for p in soup.find_all("p")]
p_combined = " ".join(p_text)

df_p = pd.DataFrame({"text": [p_combined]})

print(df_p)
```

```
                                text
0  \n Georgetown University is a private Jesuit r...
```

Question 4: Load and instantiate a spaCy NLP pipeline. Apply the pipeline to the scraped text to extract the named entities via named entity recognition (NER). Store the results in a new dataframe called ner_df with two columns: the extracted entities/text and its corresponding entity label.

Answer 4:

```
In [16]: nlp = spacy.load("en_core_web_sm")

doc_p = nlp(df_p["text"][0])

ner_df = pd.DataFrame({
    "entity_text" : [ent.text for ent in doc_p.ents],
    "entity_label" : [ent.label_ for ent in doc_p.ents]
})

ner_df.head()
```

```
Out[16]:
```

	entity_text	entity_label
0	Georgetown University	ORG
1	Washington	GPE
2	D.C.	GPE
3	United States	GPE
4	Bishop John Carroll	PERSON

Question 5: Print in descending order the top 10 most frequently mentioned people and the number of times each is mentioned. Print

in descending order the top 10 most frequently mentioned organizations and the number of times each is mentioned.

Answer 5:

```
In [17]: top_people = (
    ner_df[ner_df["entity_label"] == "PERSON"]["entity_text"]
    .value_counts()
    .head(10)
)

top_orgs = (
    ner_df[ner_df["entity_label"] == "ORG"]["entity_text"]
    .value_counts()
    .head(10)
)
```

```
In [18]: print("Top 10 People:")
print(top_people)
print("-----")
print("Top 10 Orgnaizations:")
print(top_orgs)
```

Top 10 People:

entity_text	
ROTC	3
Healy Hall	3
Barack Obama	2
Laura Chinchilla	2
Fulbright Scholars	2
Jesus	2
Dahlgren Quadrangle	2
George Tenet	2
DeGioia	2
Antonin Scalia	2

Name: count, dtype: int64

Top 10 Orgnaizations:

entity_text	
Georgetown	64
Georgetown University	13
the School of Foreign Service	6
SFS	6
NCAA	5
CIA	4
the McDonough School of Business	4
the Georgetown University Law Center	3
CSIS	3
State	3

Name: count, dtype: int64

Question 6: Do #3, #4, and #5 again but instead extract all text contained within anchor (< a >) tags on the web page. Do the results differ from the results in #5?

Answer 6:

```
In [19]: a_text = [a.get_text(strip=True) for a in soup.find_all("a")]
a_combined = " ".join(a_text)

df_a = pd.DataFrame({"text": [a_combined]})

doc_a = nlp(df_a.loc[0, "text"])

ner_a_df = pd.DataFrame({
```

```

    "entity_text": [ent.text for ent in doc_a.ents],
    "entity_label": [ent.label_ for ent in doc_a.ents]
})

top_people_a = (
    ner_a_df[ner_a_df["entity_label"] == "PERSON"]["entity_text"]
    .value_counts()
    .head(10)
)

top_orgs_a = (
    ner_a_df[ner_a_df["entity_label"] == "ORG"]["entity_text"]
    .value_counts()
    .head(10)
)

```

```

In [20]: print("Top 10 People from <a> Tags:")
print(top_people_a)
print("-----")
print("Top 10 Orgnaizations from <a> Tags:")
print(top_orgs_a)

```

Top 10 People from <a> Tags:

entity_text	
Antonin Scalia	3
Articles	3
David Malpass	2
George Washington	2
John J. DeGioia	2
Edward Douglass White	2
Patrick Francis Healy	2
James Madison	2
Martha	2
Bill Clinton	2

Name: count, dtype: int64

Top 10 Orgnaizations from <a> Tags:

entity_text	
Georgetown University	26
Georgetown	16
U.S. News & World Report	7
Hoya	6
ISSN	5
ISSN 0362-4331	4
Georgetown University Law Center	3
NCAA	3
The Washington Post Archived	3
O'Neill & Williams	3

Name: count, dtype: int64

Question 7: Briefly explain web scraping and named entity recognition/extraction in a way that a non-technical co-worker would understand.

Answer 7:

Think of web scraping like getting the data. Web scraping allows you to get the data automatically instead of having to do it by hand. It pulls the raw text and/or data from websites so it is able to be analyzed. Name entity recognition (NER) is more about making sense of the data. This process highlights the important pieces of information in text. NER finds and labels key things (people, companies, places, dates, etc.) in text.